

ARQUITETURA E GUIA DE ESTUDO

Site JD

Documentação técnica do portfólio profissional, da página pública ao painel administrativo.

React + Vite + Supabase + Netlify

Versão documentada: 1.8.1

Preparado para estudo e análise por assistentes de IA

1. Resumo executivo

O Site JD é uma aplicação de página única, conhecida como SPA (Single Page Application). A área pública apresenta perfil, especialidades, projetos, experiência, currículo e contato. A área administrativa permite alterar o conteúdo sem editar diretamente os componentes React.

O projeto usa React para construir a interface, Vite para desenvolvimento e empacotamento, CSS puro para o visual, Supabase para autenticação, banco e arquivos, GitHub para versionamento e Netlify para hospedagem.

O sistema funciona em duas camadas:

- 1. Página pública: visitantes consultam o portfolio, filtram projetos, abrem estudos de caso e enviam mensagens.
- 2. Painel administrativo: somente um usuário autenticado e cadastrado como administrador pode modificar configurações, projetos, imagens e mensagens.

O tema padrão é dark moderno, com fundo azul muito escuro, gradientes azul/roxo, glassmorphism, bordas suaves, cards arredondados e animações discretas. Aparência e classificação do site podem ser alteradas pelo painel sem substituir a arquitetura principal.

2. Tipo e formato do projeto

Item	Definição
Tipo principal	Web Application
Categoria comercial	Portfolio Website
Formato de navegação	SPA com rota principal e rota individual de projeto
Frontend	React 19 com JavaScript e JSX
Ferramenta de build	Vite 8
Estilos	CSS puro, responsivo, sem Tailwind
Backend gerenciado	Supabase
Hospedagem	Netlify
Repositório	GitHub
Banco	PostgreSQL fornecido pelo Supabase
Autenticação	Supabase Auth por email e senha
Arquivos	Supabase Storage

Por que Vite e React

React divide a tela em componentes reutilizáveis. Vite oferece início local rápido, atualização instantânea durante edição e build otimizado para produção. Essa combinação é adequada porque o site tem muita interação no navegador e não depende de renderização em servidor.

3. Tecnologias e responsabilidades

Tecnologia	Para que serve neste projeto
React	Componentes, estado, modais, filtros e painel
Vite	Servidor local, build, divisao de pacotes e manifest
Lucide React	Ícones visuais
CSS	Tema, layout, responsividade, animacoes e estados
Supabase Auth	Login real do administrador
Supabase Database	Configurações, projetos e mensagens
Supabase Storage	Capas, galeria, figurinhas e currículo
Netlify	Publicação, HTTPS, redirecionamento SPA e formulario
Git/GitHub	Histórico, branches e origem dos deploys
Node Test Runner	Testes automatizados sem framework adicional
ESLint	Análise de qualidade do JavaScript e React

4. Arquitetura geral

```
Visitante
|
v
Netlify -> index.html -> React/Vite -> componentes publicos
|
+--> Supabase: leitura de configurações e projetos
+--> Supabase RPC: envio protegido de contato

Administrador
|
v
Login -> Supabase Auth -> portfolio_admins -> politicas RLS
|
+--> edita site_settings
+--> edita portfolio_projects
+--> gerencia contact_messages
+--> gerencia portfolio-assets
```

O navegador usa somente a chave pública anônima do Supabase. A chave administrativa `service_role` não existe no frontend. A autorização real acontece nas políticas do banco, e não apenas na exibição do botão de Login.

5. Estrutura de pastas

```
Site JD/
|-- docs/                documentação e checklists
|-- public/              arquivos copiados diretamente para o build
|   |-- guide/           mini imagens do guia administrativo
|   |-- favicon.svg
|   |-- og.png           imagem de compartilhamento
|   |-- robots.txt
|   |-- sitemap.xml
|   `-- site.webmanifest
|-- scripts/             verificacoes automatizadas
|   |-- tests/           testes de validação e segurança
|   |-- check-bundle-size.mjs
|   `-- test-classifications.mjs
|-- src/
|   |-- components/      componentes publicos e administrativos
|   |   |-- admin/       componentes administrativos menores
|   |   |-- ui/          componentes genericos de interface
|   |-- data/            configurações e dados padrão
|   |-- hooks/           regras reutilizaveis de estado e autenticação
|   |-- lib/             cliente e funcoes do Supabase
|   |-- styles/          CSS global
|   |-- utils/           validação, imagens e funcoes auxiliares
|   |-- App.jsx          composição, rotas e modais
|   `-- main.jsx         ponto de entrada React
|-- supabase/
|   |-- migrations/      alterações versionadas do banco
|   `-- schema.sql       referencia completa do banco
|-- index.html           documento base e metadados
|-- netlify.toml         build, SPA e segurança HTTP
|-- package.json         versão, scripts e dependencias
|-- pnpm-lock.yaml       versoes exatas instaladas
`-- vite.config.js       configuração e divisao dos pacotes
```

6. Entrada, composição e rotas

src/main.jsx

É o primeiro arquivo JavaScript executado. Ele cria a raiz React dentro de #root, ativa StrictMode, carrega App e importa o CSS global.

src/App.jsx

É o orquestrador da interface. Ele:

- inicia os stores de site e projetos;
- acompanha a autenticação administrativa;
- aplica aparência e classificação no elemento HTML;
- decide quais seções publicas estão visíveis;
- abre modais de contato, demonstração e administração;
- interpreta a rota individual de projeto;
- carrega componentes pesados somente quando necessários.

Rotas

Endereco	Resultado
/	Página principal do portfolio
/#projetos	Ancora para a seção de portfolio
/portfolio/{id}	Página individual e estudo de caso

O projeto não usa React Router. A navegação individual utiliza `window.history.pushState`, `replaceState` e o evento `popstate`. O Netlify redireciona qualquer caminho para `index.html`, permitindo que o React interprete a rota.

7. Componentes publicos

Componente	Função
Header	Logo, menu responsivo, Login e navegação
Hero	Nome, cargo, apresentação, chamadas e box de tecnologias
LandingHighlights	Benefícios usados na classificação Landing Page
Specialties	Cards e explicações para leigos
Projects	Busca, filtros e lista de projetos
ProjectCard	Resumo visual de um projeto
ProjectDetail	Página individual e estudo de caso
ProjectModal	Demonstração, iframe, galeria, video ou link
ProjectVisual	Capa ou ilustracao padrão conforme o tipo
About	Apresentação profissional complementar
Credibility	Metricas, trajetoria, cursos e tecnologias
Resume	Currículo e Currículo Lattes
Contact	Formulario publico e chamada para contato
ContactModal	Email e redes sociais preenchidas
Footer	Identidade, links e contatos finais
SectionTitle	Título reutilizavel das seções

Os componentes `AdminPanel`, `ContactModal`, `ProjectDetail` e `ProjectModal` usam `React.lazy`. Eles não entram integralmente no arquivo principal e são baixados quando abertos.

8. Painel administrativo

O componente central e `AdminPanel.jsx`. Ele recebe os stores e a autenticação como propriedades. As areas administrativas sao:

Aba	O que altera	Reflexo no site publico
Visao geral	Resumo e atalhos	Nao altera diretamente
Configurações	Nome, cargo, textos e contatos	Header, Hero, Contato e Footer
Aparência	Paleta, fonte, estilo e preset	Tema visual completo
Classificação	Estrutura comercial do site	Organização e textos por segmento
Trajectoria e metricas	Numeros, cursos, depoimentos e ícones	Hero e Credibilidade
Box/figurinhas	Tecnologias, ícones, cores e imagens	Box lateral do Hero
Especialidades	Cards, explicações e exemplos	Seção Especialidades
Repositorios	Projetos e estudos de caso	Portfolio e paginas individuais
Mensagens	Leitura, status e exclusão	Dados do formulario, somente no painel
Guia do site	Manual visual interno	Nao altera o site publico

Cada seção pública pode ser ativada ou escondida por `sectionVisibility`. O salvamento exibe os estados: pendente, salvando, salvo ou erro.

9. Estrutura dos dados do site

As configurações principais ficam no objeto `siteConfig`, definido em `src/data/site.js` e persistido na linha `main` da tabela `site_settings`.

Grupos principais

Grupo	Exemplos de campos
Identidade	name, role, eyebrow, intro, location
Landing Page	landingLabel, landingHeadline, landingText, landingCta
Contatos	email, emailSecondary, linkedin, github, instagram, x, facebook, whatsapp, lattes
Currículo	resumeUrl, resumeSummary
Credibilidade	timelineText, testimonialsText, metrics
Tecnologias	techItems, stickerLibrary
Projetos	displayModels
Especialidades	specialties
Visibilidade	sectionVisibility
Aparência	appearance
Classificação	siteClassificationId e textos relacionados

Arrays editaveis usam um `id` estavel. Isso permite ordenar, atualizar ou excluir um item sem depender do titulo visivel.

10. Estrutura de um projeto

Cada projeto e um objeto JSON. Os campos mais importantes sao:

Campo	Significado
id	Identificador usado na URL e no banco
title	Nome do projeto
theme	Tema ou contexto
type	Comportamento tecnico base
typeLabel	Nome exibido ao visitante
displayModelId	Modelo de apresentação selecionado
presentation	Forma de demonstração
category	Área ou setor do trabalho
status	Concluído, em andamento, parado ou personalizado
description	Resumo curto do card
details	Explicacao geral
challenge	Problema enfrentado
solution	Solucao criada
results	Resultado ou impacto
duration	Prazo ou período

Campo	Significado
contribution	Participacao de Jever
audience	Publico-alvo
contentFormat	Formato da entrega
tags	Tecnologias e palavras-chave
image	Capa principal
gallery	Ate quatro imagens adicionais
embedUrl	Link incorporado, como Power BI
externalUrl	Link externo do sistema ou material
videoUrl	Video demonstrativo
featured	Define se aparece no portfolio publico
accent	Cor de destaque

Tipos base

- `powerbi`: dashboard incorporado;
- `website`: sistema ou site ao vivo;
- `content`: conteúdo digital, documento ou galeria;
- `ai`: GPT ou agente de IA básico.

Os modelos personalizados reutilizam um comportamento base. Assim, um novo nome de modelo não exige criar uma nova lógica de abertura.

11. Estado e persistencia

`useSiteStore`

Gerencia configurações gerais. Le primeiro o cache local, consulta `site_settings`, normaliza o objeto e salva alterações com atraso de 700 ms. Em falha, tenta novamente ate tres vezes e mantem a copia local.

`useProjectStore`

Gerencia a lista de projetos. Salva somente os registros alterados, agrupa digitacoes rapidas, tenta novamente em falhas e atualiza posicoes ao reordenar. Ao excluir um projeto com sucesso, remove os arquivos proprios associados.

Armazenamento local

`localStorage` funciona como cache e modo local de desenvolvimento. Ele não substitui o Supabase para sincronização entre dispositivos. As chaves principais são:

- `jd-portfolio-site-v1`;
- `jd-portfolio-projects-v1`;
- `jd-contact-last-submit`.

12. Banco de dados Supabase

Tabela	Conteúdo	Leitura pública	Escrita
portfolio_admins	UUIDs autorizados	Nao	Administrativa
site_settings	Objeto geral do site	Sim	Somente administrador
portfolio_projects	Objetos dos projetos	Apenas destacados	Somente administrador
contact_messages	Mensagens recebidas	Nao	RPC pública controlada e administrador

Storage

O bucket `portfolio-assets` e publico para leitura, mas upload, alteração e exclusão exigem usuário autenticado presente em `portfolio_admins`. Tipos permitidos: PNG, JPEG, WebP, GIF e PDF. O limite e 10 MB por arquivo.

Migrações

- `202607150001_portfolio_initial.sql`: estrutura inicial, tabelas, RLS e Storage;
- `202607160001_security_hardening.sql`: formulario protegido, limites e retirada do INSERT publico direto.

13. Login e segurança

O fluxo real de Login e:

1. Email e senha sao enviados ao Supabase Auth.
2. O Supabase devolve uma sessao somente se as credenciais forem validas.
3. O frontend consulta `portfolio_admins` para verificar se o UUID tem autorizacao.
4. Se não estiver autorizado, a sessão é encerrada.
5. Mesmo com uma sessao, as politicas RLS repetem a verificacao em cada escrita.

O PIN local existe apenas quando o Vite está em modo de desenvolvimento, o Supabase está desativado e `VITE_ADMIN_PIN` foi preenchido. Ele não é um mecanismo de produção e não aparece no pacote publicado.

Controles implementados

- RLS nas tabelas;
- allowlist por UUID;
- ausencia de `service_role` no frontend;
- validação de URLs HTTP/HTTPS;
- limites de tamanho em texto, backups e arquivos;
- nomes de arquivo e pasta normalizados;
- formulario por função RPC `security_definer` com validação;
- intervalo minimo de um minuto por email;
- limite de cinco mensagens por hora;
- bloqueio de mensagem repetida;
- honeypot no formulario Netlify;
- CSP, HSTS, bloqueio de iframe externo do proprio site e restricoes de permissao;
- iframe de demonstração com `sandbox`;
- dependencias com versoes fixas.

Nenhum sistema conectado a internet pode ser declarado impossível de atacar. A proteção depende também de senha forte, email protegido, revisão das configurações do Supabase e atualização periódica das dependências.

14. Formulário e notificações

O formulário valida os dados no navegador e no banco. A mensagem é enviada para a função `submit_contact_message`. Em produção, o mesmo envio é preparado para o Netlify Forms, permitindo notificação por email quando o destinatário for configurado em `Forms > Form notifications`.

O painel consulta `contact_messages` e permite marcar como lida, arquivar ou excluir.

15. Imagens e arquivos

`optimizeImage.js` reduz dimensões exageradas e tenta converter PNG/JPEG para WebP. A conversão é usada somente se o resultado for menor. O tamanho visual do card não muda; o arquivo transferido fica mais leve.

Ao substituir ou remover uma imagem hospedada no bucket do próprio projeto, o sistema tenta eliminar o arquivo antigo para evitar acúmulo. URLs externas não são excluídas.

16. CSS, classes e responsividade

O projeto usa um único arquivo principal: `src/styles/global.css`. Não existem CSS Modules nem Tailwind. As classes seguem uma convenção próxima de BEM:

```
.project-card { }           /* bloco */
.project-card__content { }  /* elemento interno */
.project-card--featured { } /* variacao */
.is-success { }            /* estado */
```

Padrões usados:

- `__` separa uma parte interna do componente;
- `--` representa uma variação visual;
- `is-` representa um estado temporário;
- `admin-` identifica o painel;
- `modal-` identifica diálogos;
- `data-site-classification` altera estrutura por classificação;
- variáveis CSS controlam cores, fontes, fundos e acentos.

O layout usa Grid e Flexbox. Media queries reorganizam menus, cards, painel e formulários em tablet e celular. `:focus-visible` destaca navegação por teclado e `prefers-reduced-motion` reduz animações quando solicitado pelo sistema.

17. Acessibilidade

Os modais usam `role="dialog"`, `aria-modal`, título associado e controle de foco. O hook `useModalA11y`:

- envia o foco para o modal;
- mantém Tab e Shift+Tab dentro do diálogo;
- fecha com Escape;
- devolve o foco ao elemento que abriu o modal;
- bloqueia a rolagem da página por trás.

O site possui link para pular ao conteúdo, rótulos acessíveis, mensagens com `aria-live` e suporte a redução de movimento.

18. Classificação, aparência e arquitetura

Esses conceitos são separados:

- Classificação: define a finalidade e reorganização básica, como portfolio, landing page, site institucional, profissional autônomo, evento ou blog simples.
- Aparência: define paleta, fonte, estilo e acabamento.
- Arquitetura: permanece React/Vite com os mesmos dados e componentes centrais.

Mudar a classificação não deve ser usado apenas para trocar cores. Mudar a aparência não deve apagar nomes, projetos ou contatos.

19. Build e desempenho

O Vite gera a pasta `dist`. O build utiliza divisão manual de pacotes:

Pacote	Conteúdo aproximado
index	Código principal do site, cerca de 86 KB
vendor-react	React e React DOM
vendor-supabase	Cliente Supabase
vendor-icons	Ícones Lucide
AdminPanel	Painel carregado sob demanda
ProjectDetail	Página individual sob demanda
ProjectModal	Demonstração sob demanda
ContactModal	Redes sociais sob demanda

O teste `check-bundle-size.mjs` falha se o pacote principal ultrapassar 150 KB ou perder a divisão dos fornecedores.

20. Netlify e publicação

Configuração atual:

- comando: `npm run build`;
- pasta publicada: `dist`;
- redirecionamento SPA: qualquer rota retorna `index.html`;
- deploy de produção ligado a branch `main`;
- headers de segurança definidos em `netlify.toml`;
- assets versionados recebem cache de um ano.

A branch de desenvolvimento é `develop`. A `main` deve receber alterações somente quando houver autorização, pois cada atualização pode consumir créditos de build do Netlify.

21. Git e versionamento

Regra SemVer adotada:

- MAJOR: mudança grande ou incompatível, exemplo 1 para 2;
- MINOR: nova funcionalidade compatível, exemplo 1.7 para 1.8;
- PATCH: correção pequena, exemplo 1.8.0 para 1.8.1.

Fluxo recomendado:

```
develop -> testar localmente -> commit -> Pull Request -> main -> Netlify
```

Commits seguem Conventional Commits, como:

```
feat(admin): adiciona nova função
fix(contact): corrige envio de mensagem
perf(build): divide pacote principal
docs(architecture): documenta o projeto
```

22. Comandos de desenvolvimento

No CMD do Windows:

```
cd /d "C:\Users\jever\Documents\Site JD"
pnpm install
pnpm run dev -- --host 127.0.0.1 --port 4173
```

Abrir no navegador: <http://127.0.0.1:4173>.

Comandos de verificacao:

```
pnpm run lint
pnpm run test:all
pnpm run build
pnpm run check:bundle
```

Tambem existem `iniciar-site.cmd`, `abrir-vscode.cmd` e o workspace `Site JD.code-workspace`.

23. Variáveis de ambiente

```
VITE_SUPABASE_URL=URL_PUBLICA_DO_PROJETO
VITE_SUPABASE_ANON_KEY=CHAVE_ANON_PUBLICA
VITE_ADMIN_PIN=PIN_APENAS_LOCAL
VITE_FORCE_LOCAL_ADMIN=true # somente teste local controlado
```

Arquivos `.env` não devem entrar no Git. A chave anônima pode existir no navegador; sua segurança depende das políticas RLS. Chaves administrativas nunca podem usar o prefixo `VITE_`.

24. Testes existentes

Teste	Cobertura
validation.test.mjs	URLs, projetos, backups e formulario
security.test.mjs	dependencias, headers, RPC, PIN e segredos
test-classifications.mjs	oito classificações em desktop e celular
check-bundle-size.mjs	tamanho e divisao do JavaScript

Na versão documentada: lint aprovado, 12 testes unitários aprovados, 8 classificações aprovadas, build aprovado e resposta local HTTP 200.

25. Como pedir uma explicacao ao GPT

Envie este documento e use um pedido como:

```
Análise esta documentação do Site JD. Explique o projeto como um professor,
do básico ao avançado. Primeiro apresente arquitetura e fluxo de dados.
Depois explique cada pasta, componente, hook, classe CSS, tabela do Supabase,
regra de segurança, build e deploy. Use exemplos simples e crie uma ordem de
estudo para eu aprender a manter o projeto sem quebrar a produção.
```

Para estudar um arquivo específico:

Com base na documentação, explique o arquivo `useProjectStore.js` linha por linha em linguagem simples. Mostre como o `debounce`, a fila, as tentativas de salvamento e a persistência local trabalham juntos.

26. Pontos de atenção e próximas melhorias

- Trocar a senha temporária antes da abertura definitiva.
- Configurar notificação de email no Netlify Forms.
- Testar o site com Lighthouse e leitores de tela.
- Reduzir e organizar o CSS global em módulos por domínio.
- Avaliar carregamento tardio do cliente Supabase para reduzir a primeira visita.
- Cadastrar projetos, links e imagens reais.
- Revisar periodicamente dependências e políticas RLS.
- Manter a `main` protegida e publicar somente versões validadas.

27. Mapa rápido de manutenção

Quero alterar	Arquivo ou área principal
Composição da página	<code>src/App.jsx</code>
Textos e valores padrão	<code>src/data/site.js</code>
Projetos padrão	<code>src/data/projects.json</code>
Tipos de projeto	<code>src/data/projects.js</code>
Classificações	<code>src/data/siteClassifications.js</code>
Paletas, fontes e estilos	<code>src/data/appearance.js</code>
Visual geral	<code>src/styles/global.css</code>
Login	<code>src/hooks/useAdminAuth.js</code>
Salvamento do site	<code>src/hooks/useSiteStore.js</code>
Salvamento de projetos	<code>src/hooks/useProjectStore.js</code>
Supabase e arquivos	<code>src/lib/supabase.js</code>
Validação	<code>src/utils/validation.js</code>
Banco e RLS	<code>supabase/schema.sql</code> e migrations
Build e pacotes	<code>vite.config.js</code>
Netlify e segurança HTTP	<code>netlify.toml</code>

Este documento descreve a arquitetura conhecida na versão 1.8.1. Antes de aplicar uma orientação automática, compare a documentação com o código atual, trabalhe na branch `develop`, execute os testes e não envie para `main` sem revisar o impacto no Netlify.